

Stock Sim - Historical Market Analyzer & Trader Strategy Simulator

Jason Ma, Pei Ma

ESE 224 Final Project

Stony Brook University

Contents

1	Introduction	2
1.1	Problem Statement	2
1.2	Motivation	2
1.3	Dataset Description	3
2	Data Structure Design	3
2.1	Linked List Design	4
2.2	Binary Search Tree Design	4
2.3	Queue Design	5
2.4	Circular Queue Design	6
2.5	Stack Design	7
3	Trading Strategies	8
3.1	Fixed SIP Strategy	8
3.2	Dynamic SIP Strategy	8
3.3	Golden Cross Strategy	9
3.4	Momentum Strategy	10
4	Results and Analysis	10
4.1	Backtesting on SPX	10
4.2	Comparsion Table based on SPX	11
4.3	Backtesting Results	11
4.3.1	Fixed SIP Strategy	11
4.3.2	Dynamic SIP Strategy	11
4.3.3	Golden Cross Strategy	12
4.3.4	6-Month Momentum Strategy	12
4.4	Dynamic vs Fixed SIP	12
4.5	Volatile NVDA vs SPX	13
5	Conclusion	13
5.1	Lessons Learned	13
5.2	Limitations	14
5.3	Improvements	14

1 Introduction

1.1 Problem Statement

The goal of this project is to design and implement a C++ command line stock market backtesting program which loads, stores, and analyzes historical stock market data. The system is built using custom data structures to simulate four different investment strategies: Fixed SIP, Dynamic SIP, Golden Cross, and Momentum. With a custom Dynamic SIP strategies, this begs the question:

“Does a Dynamic SIP (invest more on dips) outperform a Fixed SIP over
25 years of S&P 500 data?”

In order to answer this question, this program will load historical stock data and simulate a comparison between different investment strategies and compare their overall performance. So, each strategy is designed to produce will produce the following performance values:

- Final portfolio value,
- Total Investment,
- Total Return,
- Compound Annual Growth Rate (CAGR),
- Max drawdown,
- Total Trades.

1.2 Motivation

The motivation behind the creation of this simulation program is to address the above question and demonstrate the effectiveness of using trading strategies to maximize returns. Specifically, we aim to clarify whether increased investing on large dips in a stock can actually help with increasing earnings over time (dynamic vs. fixed SIP strategies). Intuitively, this dynamic investment strategy may seem to be the improved investment strategy but due to market volatility it is not guaranteed to outperform a fixed investment strategy. Not only to answer this question on financial analysis, this project also serves to implement core programming data structures that include

- Linked lists,
- Queues,

- Stacks,
- Binary search trees,
- Vectors.

These structures will prove important as data management, average calculations, order processing, and more will rely on using these structures for organization and storage of trade records.

1.3 Dataset Description

This simulation program utilizes historical stock data stored in .csv formatted files. This program extracts the data from the .csv files and deposits the information into a custom doubly linked list implemented in price history, where iterators can be used to traverse through the data. From this point, this data can be used to backtest all the trading strategies implemented in the program. The data provided in the .csv files are:

- Date
- Open
- High
- Low
- Adj Close
- Trade Volume

And covers an extensive period of trading, starting around the year 2000. This gives a variety in terms of market behavior, where high volatility, crashes, bull and bear markets, are all present in this 20+ year span of data.

2 Data Structure Design

The Stock Simulation project required the usage of several custom written data structures to efficiently examine stock price history, trading operations, and simulate investment strategies. Each structure This project implemented doubly linked lists, circular queues, first-in-first-out (FIFO) queues, stacks, and binary search trees in C++ instead of the STL containers.

2.1 Linked List Design

The doubly linked list is implemented within the price history class which consists of price nodes. The linked list allows for storage of historical stock price data in chronological order. Each node consists of information of a stock within a certain day and important node variables to assist the doubly linked list functions. Thus, the variables in each node are the following: **date**, **open** (price), **high** (price), **low** (price), **close** (price), **volume** (trading volume of a day), **next** (node), **prev** (node). The linked list class utilizes head and tail nodes to help with traversal and also size to keep track of the extensive history length. It supports sequential insertion, forward and reverse traversal as it includes iterators within, as well as date searching and date range printing support utilizing linked list traversal.

Time Complexity:

- **append()** would have time complexity $O(1)$. This is because to append the process simply inserts a node previous to the tail.
- **findByDate()** would have time complexity $O(n)$ since in the worst case it has to travel through every single node to find the specific instance
- **printRange()** has time complexity $O(n)$ since it will iterate through all history node sequentially
- Forward and Reverse Traversals with the iterators would be $O(n)$ due to similar reasons above (sequentially travels through all nodes)
- **getSize** has time complexity $O(1)$ since it simply returns the value of size member variable

This structure is a good choice for its application because doubly linked lists can naturally grow as nodes are included sequentially, which is also convenient due to its tail node. Traversals are also easy to implement using iterators as traversals can be done in both forward and reverse directions.

2.2 Binary Search Tree Design

The binary search tree is implemented in the Stock BST class. It allows for organization of stocks by annual return percentage. Leftwards insertion is for lesser values, while rightwards is for greater. Each node in the tree stores information about stock ticker, annual return, year, and left and right child pointers. Due to this being a tree, it contains a root node. It also contains the following functions that help with traversal and printing the information contained in the tree.

Time Complexity:

- `insert(const string& ticker, double key, int year)` has average $O(\log n)$ time complexity because each recursive call splits the BST into approximately half its values. The worst case scenario is $O(n)$ if the tree becomes unbalanced, which is if values are sorted in one direction from the root. For example, any value in the successive subtree is greater than its root, such as 10-20-30-40-50, a tree that branches towards its right side.
- `search()` has $O(\log n)$ time complexity due to the same reasoning as above (recursive style tree traversals)
- `rangeSearch()` has time complexity $O(\log n + k)$, since unrelated branches are ignored and only valid values are traversed.
- Traversal algorithms (`inorder()`, `preorder()`, `postorder()`) all have $O(n)$ time complexity, since each node is visited once.
- `getHeight()` has time complexity of $O(n)$ since it recursively checks each subtree, and looks through all nodes

This structure is a good choice for its application (organized by value), since entire branches of the BST can be ignored while performing searches, making the operation much quicker and more efficient. This also makes range searches easy as well, since again different branches can be excluded from the search due to the ordering of the BST. When nodes are added into the BST, the order is already well sorted, as a result of the insertion criteria of ($\text{key} < \text{node} \rightarrow \text{key}$) conditional which places lesser annual return nodes to the left, and greater to the right. The BST, like the doubly linked list, can grow dynamically as well, on top of maintaining ordering.

2.3 Queue Design

The OrderQueue class is implemented using a singly linked list (SLL) based queue. Each node in the queue, represented by the QueueNode struct, contains the next QueueNode as per the SLL structure and Order data (a struct containing information on the ticker, type, decision, target price, shares, and order date of an order).

The queue is valuable to simulate ordering processes such as BUY and SELL orders on stocks. Oldest orders are given the priority while newer orders follow, which is exactly what the queue data structure supports.

The queue consists of the front node and back node to ensure efficient insertion and removal operations specific to the FIFO behavior.

Time Complexity:

- `enqueue(const Order& order)` has complexity $O(1)$. This only appends a new node to the already referenced back pointer.
- `dequeue()` has complexity $O(1)$. Similarly to above, it references front and removes the front most node (or the oldest node in the queue)
- `Peek()` has complexity $O(1)$. It only examines the data at the front of the queue, which is referenced using the front pointer.
- `isEmpty()` has complexity $O(1)$. Returns a boolean referencing size, a member variable in the queue class.
- `getSize()` is similar to above, and is also $O(1)$. It returns the value of size, a member variable.
- `printAll()` has complexity $O(n)$, since it must traverse through all nodes in the singly linked list queue.

The advantage of using a singly linked list queue for this application is that the size is dynamically adjusted and is not capped, which is useful for accommodating any amount of orders. A queue in general is also analogous to how trades are conducted in reality (FIFO). Using additional front and back pointers also made the accessibility and usability of the queue much more efficient and apparent as well for storing and accessing orders.

2.4 Circular Queue Design

The Golden Cross trading strategy relies on calculating moving averages over a rolling time window; typically a short-term window of 50 days and a long-term window of 200 days is used. It would be intuitive to collect a sequence of data until we reach the desired quantity of information. Then, if more information is needed, newer data will replace older data.

A circular queue is chosen to support this strategy because it has a FIFO behavior and uses a fixed size array. This implementation uses an array-based structure with head, tail, count, and capacity variables to track the queue state. The head and tail variables represent the indices of the front and back of the queue. If the indices reach the end of the array, the value will wrap around using modular arithmetic. Additionally, we also incorporated a sum variable for better time complexity when calculating the moving averages for the Golden Cross strategy.

It features a head and tail which aids with insertion and wrap-around behavior, which is required due to the nature of this queue being used for moving averages.

Time Complexity:

- `enqueue(double value)`, `dequeue()`, and `peek()` are all $O(1)$ time complexity. Using head and tail, direct access is supported so insertion and deletion is made quickly, as well as observing the front using peek.
- `getAverage()` is $O(1)$ time complexity as it computes an average using a running sum stored in a private variable `sum` and returns that value. In a traditional circular queue implementation, calculating the average would require traversing through a fixed-size array resulting in $O(n)$ time complexity. By updating `sum` when elements are added and removed, the average can be computed in one step, reducing the operation to constant time.
- `isFull()` and `isEmpty()` are $O(1)$ since they perform a comparison with the count and the max or min values that can be stored in the array buffer.

The circular queue is the ideal data structure for the Golden Cross strategy since the moving averages depend on a fixed size rolling set of recent prices. As new stock prices are added to the queue, the oldest prices get removed. The circular queue also supports this behavior with minimized time complexity. Its constant time operations makes the strategy very effective for large quantity analysis of stocks.

2.5 Stack Design

The `TradeStack` class uses the stack data structure. It serves as a record for each trade executed, as it stores them chronologically (most recent trade on the top of the stack). It utilizes a singly linked list like the queue, each node containing information about the trade in the form of Trade Record (stores ticker, date, price, shares, the action buy or sell, and the total cost of the trade). Each node also contains a next as to serve to link the separate transactions together into one record. Finally, a top pointer is used which is useful for quick access to the most recent trade.

Time Complexity:

- `push(const TradeRecord& record)`, `pop()`, and `peek()` operations all have time complexity of $O(1)$. They directly reference off of the top pointer to perform their operation (either insert, remove, or observe the node data at the top of the stack)
- `isEmpty()` has complexity $O(1)$. Simply check with a condition the size, a member variable, is 0.

- `getSize()` has complexity $O(1)$, since it returns the value of `size`, a member variable
- `printAll()` has complexity $O(n)$, since it prints the entire stack so every node is visited.

Using a stack for this purpose is the best choice for this application, as it automatically stores the history very intuitively and provides easy access to the most recent trade for modification. For this reason, implementation of a trade undo function is simple to implement due to this access. The application of a singly linked list to store the stack's nodes is useful as this means the stack can grow dynamically without hitting a storage limit. Traversal through the stack for printing can also be done easily due to this singly linked list core in the stack implementation.

3 Trading Strategies

3.1 Fixed SIP Strategy

The approach of the fixed SIP strategy is to invest a fixed amount into the market, regardless of the conditions of the market at that time. This is the baseline strategy that will be used as a reference for comparing all other strategies implemented. In the program, the required parameters are monthly capital, beginning of investment, and end of investment in years (as well as the price history data that the strategy will use to traverse through and invest off of)

In the simulation program, the strategy traverses through price history (linked list) in order to find the occurrence of a new month. At that instant, a fixed amount is then invested into the market, using the monthly capital to purchase as many shares as possible in the month ($\# \text{ of shares} = \text{Monthly capital} / \text{Closing price}$). With this transaction, updates are done to the total shares owned and capital invested. Portfolio value is updated monthly by taking shares and multiplying by the closing price at the instant. At the end of the program, the final computations are performed to report back CAGR, max draw down, portfolio value, and reporting the total trades executed.

We expect that this strategy will allow for the overall portfolio value to grow steadily as the algorithm automatically invests more in periods of decline, and less in expensive periods as a result of constant monthly investment.

3.2 Dynamic SIP Strategy

The Dynamic SIP Strategy is an adaptive strategy which adjusts the monthly investment based on market conditions, rather than investing a fixed amount of money each period.

Unlike the traditional fixed SIP, which invests the same amount of money each month, this strategy invests based on market indicators such as momentum, trends, and volatility.

This strategy first determines the initial budget over the backtesting time period which is computed by multiplying the number of months by the monthly capital. This ensures the strategy does not over-invest and is comparable to the fixed SIP contribution. During the simulation phase, the strategy evaluates the market based on three indicators:

- 12-month momentum,
- 12-month moving average trend, and
- 12-month volatility.

Based on these indicators, the market is classified into several types: deep bear, bear, neutral, bull, and strong bull. Given the market type, the strategy may apply a multiplier to the monthly capital to invest a varying amount. The actual investment is the minimum between the multiplied investment and the remaining budget, to prevent overspending. The multiplier for each type may be adjusted in the code, and may work differently for different stocks.

3.3 Golden Cross Strategy

The golden cross strategy is based on moving average crossover analysis, and at each crossover event the results of the moving averages determine a buy or sell action. The purpose of the strategy is to identify the most probable events of upward momentum in the market, where investment would be most profitable. It does not invest at regular periods, but only based on the indicators given by the strategy (at crossover events with the correct conditions). The required parameters for backtesting are monthly capital, beginning of investment, and end of investment in years (as well as the price history data that the strategy will use to traverse through and invest off of).

In the program there are two circular queues that will be used to calculate moving averages between 50 day and 200 day periods. The 50 day is used for recent market behavior, while the 200 day represents long term market trend. The buy signal is given when the moving average of the 50 day is greater than that of the 200 day, indicating upward market momentum. Conversely, if the 50 day moving average is lesser than that of the 200 day, this is a sell signal that indicates unfavorable market trajectory for investment. At the end of the program, the final computations are performed to report back CAGR, max draw down, portfolio value, and reporting the total trades executed.

We expect this strategy to perform when the market experiences stable trends in which the benefit of using crossover events can be most profitable. However, in more volatile

markets where fluctuations are common, these crossover events may signal an action too prematurely and could miss out on investment opportunities that the volatility may bring.

3.4 Momentum Strategy

The momentum strategy is an investment approach that allocates capital based on recent price performance, under the assumption that the assets will follow the trends: rising prices will tend to rise more and falling prices will tend to fall. In the designed backtest, the momentum strategy analyzes a 6 month period and measures past returns over that window. If the momentum is positive and is above a chosen threshold, the strategy invests; if negative, the strategy does not.

The core logic of the momentum backtest strategy is to compute rolling performance over a fixed lookback window, which is of 6 months for this project’s design. At each monthly interval, the strategy compares the current asset price with the price from 6 months earlier to calculate momentum value:

$$M_t = \frac{P_t - P_{t-6}}{P_{t-6}}$$

where P_t represents the current price and P_{t-6} represents the price 6 months earlier. The value M_t represents the percentage return over the previous 6 months and acts as a momentum signal used for investment decisions.

If the calculated momentum exceeds the threshold, the strategy interprets this as a positive trend and invests the entire monthly capital contribution into the asset, but if it fails, then the strategy does not invest any capital. This allows the strategy to potentially reduce losses during bear markets and improve returns during strong bull markets.

We expect this strategy to perform best during the decade-long bull market, since that is when prices are increasing. The strategy will likely be less active during the dot-com crash (2000-2002) or financial crisis (2008).

4 Results and Analysis

4.1 Backtesting on SPX

To reiterate, this project incorporates the four backtesting strategies: Fixed SIP strategy, Dynamic SIP, Golden Cross, and (6-Month) Momentum strategy. Performance is measured based on final portfolio value, total return, compound annual growth rate (CAGR), maximum drawdown, and total number of trades.

The final portfolio value is the total value of the portfolio at the end of the simulation period, which includes all invested capital, all gains and losses from trades, and compounding

over time. The total invested is the sum of money contributed to the strategy over time. The total return is the percent change from the total invested to final value. The CAGR is the average yearly growth rate assuming consistent compounding over time. The max drawdown is the largest percent drop before reaching a new peak in the portfolio during a single period. Finally, the number of trades should be understood as it is written.

The following results for each backtesting strategy tested using SPX's historical stock data from Yahoo Finance CSV files from 2000 to 2020 with a monthly capital of \$1,000. To clarify, monthly capital is synonymous to base contribution amount, so it is not necessarily required to spend that exact amount each month.

4.2 Comparision Table based on SPX

Metric	Fixed SIP	Dynamic SIP	Golden Cross	Momentum
Final Value	\$553,772.26	\$727,653.89	\$521,001.42	\$281,069.80
Total Invested	\$240,000.00	\$240,000.00	\$240,000.00	\$136,000.00
Total Return	130.74%	203.19%	117.08%	106.67%
CAGR	4.50%	5.70%	3.95%	3.70%
Max Drawdown	46.25%	39.81%	16.92%	54.32%
Total Trades	240	114	17	163

Table 1: Performance comparison of investment strategies for SPX from 2000-01-01 to 2020-01-01 with an monthly capital of \$1000.

4.3 Backtesting Results

4.3.1 Fixed SIP Strategy

The Fixed SIP Strategy strategy consistently invested a fixed \$1,000 every month regardless of the market conditions. Over 251 trades, this strategy produced a final portfolio value of \$602,377.04 with a total invested capital of \$251,000. The total return was 139.99%, and the CAGR was 4.47%. The strategy also experienced a large maximum drawdown of 46.25%, reflecting a large decline period.

4.3.2 Dynamic SIP Strategy

The Dynamic SIP Strategy adjusted monthly investment amounts based on market trends. During market declines, the strategy increased investment contributions and if the market expands, then the strategy decreased investment contributions. This method approach is meant to purchase more shares at lower prices and fewer shares at higher prices.

The strategy achieved a final portfolio value of \$701,598.76 with a total invested capital of \$308,900.00. The total return was 127.13%, and the CAGR was 3.96%. This strategy, like the Fixed SIP strategy, also experienced a large maximum drawdown, but of 51.37%.

4.3.3 Golden Cross Strategy

The Golden Cross Strategy used moving averages to determine when to invest in the market. The strategy decided to buy when short-term moving average rose above the long-term moving average and avoided buying during falling prices.

In a total of 19 trades, this strategy produced a final portfolio value of \$475,845.15 with a total invested capital of \$252,000.00. The total return was 88.83% and the CAGR was 3.07%. Compared to the other strategies, the Golden Cross strategy had the lowest maximum drawdown of 33.20%.

4.3.4 6-Month Momentum Strategy

The 6-Month Momentum strategy invests only when the market shows positive momentum over the previous six months. If the market showed negative momentum over the previous six months, the strategy would not invest.

In a total of 170 trades, this strategy produced a final portfolio value of \$305,321.18 with a total invested capital of \$142,500.00. The total return was 114.26% and the CAGR of 3.70%. The 6-Month Momentum strategy had the largest maximum drawdown of 54.32%, indicating that this strategy is not always effective at avoiding major market drops. Overall, the 6-Month Momentum strategy performed the worst amongst the four strategies in this portfolio.

4.4 Dynamic vs Fixed SIP

The Fixed SIP and Dynamic SIP strategies both invested the same total amount of \$240,000 over the backtesting period between 2000-01-01 and 2020-01-01 (\$1000 for each 240 months). Although both strategies invested the same total amount of capital (\$240,000), the Dynamic SIP achieved a final portfolio value of \$727,653.89 compared to \$553,772.26 for the Fixed SIP strategy, representing approximately 31.4% higher portfolio growth. The Dynamic SIP strategy also produced a higher CAGR of 5.70% versus 4.50% for Fixed SIP, an improvement of roughly 26.7%. In terms of risk, the Dynamic SIP reduced maximum drawdown from 46.25% to 39.81%, lowering downside risk by about 13.9%.

However, the strategy may not always outperform in rapidly changing markets where trend signals become less reliable. In more stable market environments, the advantage over Fixed SIP may become smaller. The Dynamic SIP strategy was customized to take advantage

of bear markets, taking advantage of the dot-com crash in 2000-2002 and the financial crisis in 2008. So, this strategy was able to make the most profit because it was taking advantage of recessions and allocated most of the budget for the unfortunate time periods.

4.5 Volatile NVDA vs SPX

Metric	Fixed SIP	Dynamic SIP	Golden Cross	Momentum
Final Value	\$14,209,327.97	\$10,994,320.55	\$3,273,278.65	\$6,266,878.93
Total Invested	\$252,000.00	\$267,600.00	\$252,000.00	\$169,500.00
Total Return	5538.62%	4008.49%	1198.92%	3597.27%
CAGR	22.34%	19.36%	12.99%	18.76%
Max Drawdown	56.09%	63.64%	47.25%	83.62%
Total Trades	252	252	21	175

Table 2: Performance comparison of investment strategies for AMZN from 2000-01-01 to 2020-01-01 with an monthly capital of \$1000.

In comparison to the backtest results for the SPX stocks, there were much higher returns for AMZN stocks due to Amazon’s stronger long-term growth between 2000 and 2020. However, the higher returns on AMZN also came with higher volatility.

For SPX, the Dynamic SIP strategy achieved the best overall performance, outperforming Fixed SIP strategy by approximately 31.4% in final portfolio value. It also reduced maximum drawdown from 46.25% to 39.81%.

For AMZN, the Dynamic SIP strategy also achieved best overall performance, outperforming Fixed SIP strategy greatly by a 179% higher final portfolio value. However, the maximum drawdown was much greater for the Dynamic SIP strategy compared to the Fixed SIP strategy, indicating that Amazon stock had much higher volatility.

The Golden Cross strategy delivered the weakest results, with a 749% total return, second to the Momentum strategy’s results of 2082.77% total return. This performance is likely due to the conservative nature of the Golden Cross strategy as they had only completed 20 total trades.

5 Conclusion

5.1 Lessons Learned

One of the primary lessons learned from the creation of the stock simulation program was about how to decide which data structures would be used to fulfill what purpose. We were

able to realize that each application, with their own requirements for organization, meant that different data structures would be more suited for the implementation. For example, in order queue it was apparent that a queue would be the best data structure to implement, due to the nature of pending orders and addressing orders in the sequence they came. A less obvious example however would be the application of the binary search tree, which proved its worth in how it was able to organize based on the key value of annual return percentage of stocks, and search relatively quickly due to its branching structures and through recursion.

Another important lesson learned from the project was how important it was to have an organized interface after the development of all the major classes. We were able to see that creating the `main.cpp` file was much more intuitive, due to the fact that many of the core functions of the stock simulation were already implemented through the files.

5.2 Limitations

A major limitation of the stock simulation program is that it is idealized in comparison to reality. This simulation only deals with a scenario where strategies and their success are measured only on the closing prices, which are the only relevant fee that is present during the trading.

In reality there would be additional fees that arise from commissions when a trade is done, taxes (which would be impactful for frequent trading), and more. So the simulation is far more optimistic in its results and doesn't account for the external fees that could affect the actual total returns for each of the strategies.

5.3 Improvements

A possible improvement to the program above could be to address the additional fees issue and make the simulation more realistic in its results for the strategy comparisons and executions. Implementation of trading effects like commissions upon trade executions could make the results more reflective of stock investments in the real world.

Another improvement for the overall program could be to implement more safety checks and validations for proper input structure. For example, the main menu still relies on the user inputting properly formatted input messages for the program to run properly. Employing more validation tests may be helpful in order to avoid program bugs or crashing from non-conforming inputs.